



天津大学
Tianjin University



Harpagon: Minimizing DNN Serving Cost via Efficient Dispatching, Scheduling and Splitting

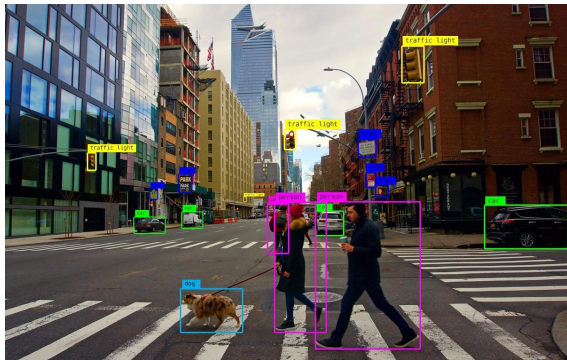
Zhixin Zhao¹, Yitao Hu¹, Ziqi Gong¹, Guotao Yang¹,
Wenxin Li¹, Xiulong Liu¹, Keqiu Li¹, Hao Wang²

zhixin612cs@gmail.com

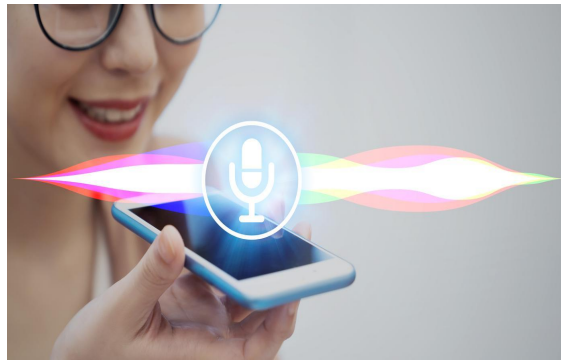
¹ Tianjin Key Laboratory of Advanced Networking, Tianjin University

² Stevens Institute of Technology

DNN has revolutionized multiple fields



Computer Vision



Speech Recognition

Application developer, cloud and clients

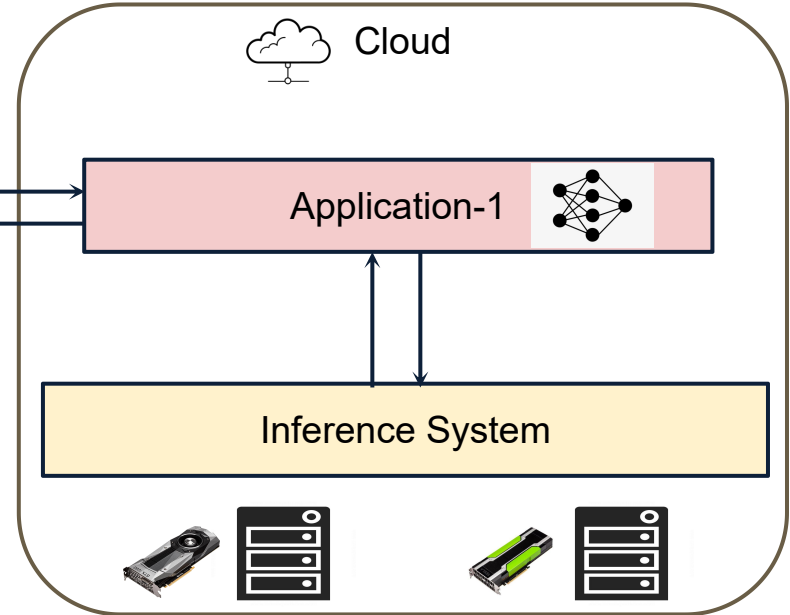
Application Developer



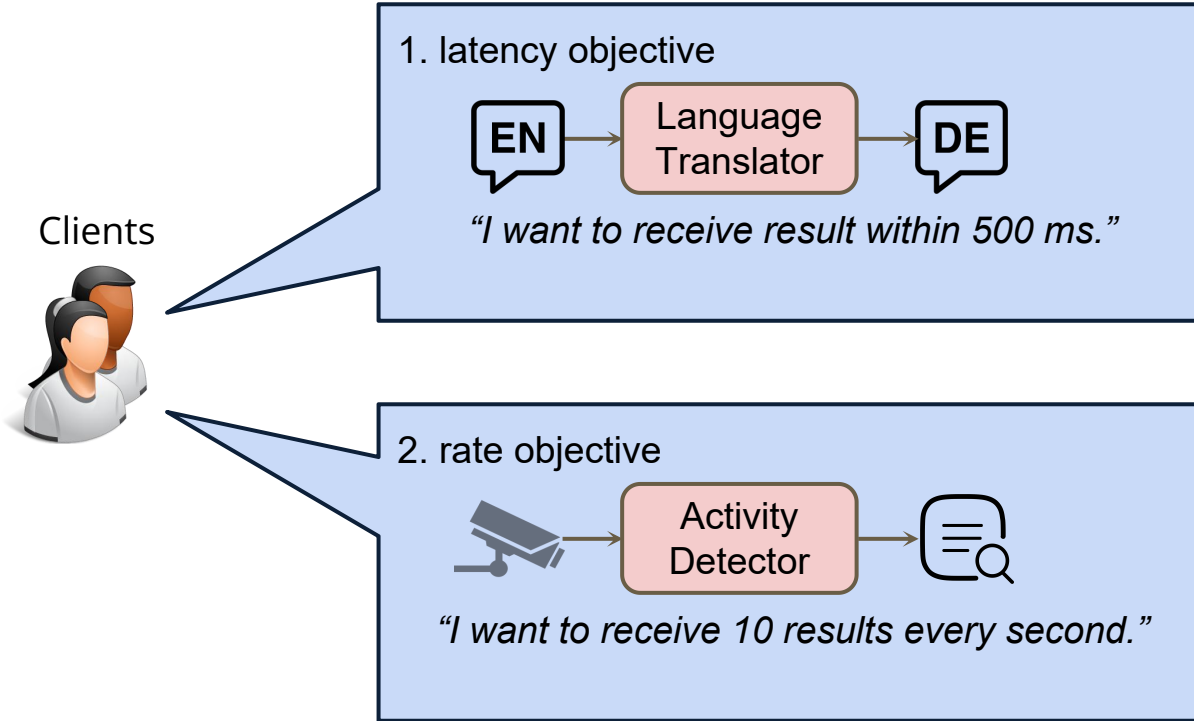
Clients



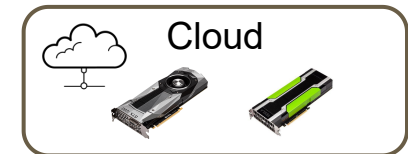
Cloud



Clients have performance objectives



Application Developer



Design goal

Clients

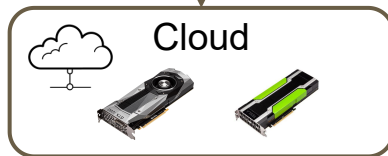


1. latency
2. rate

Application Developer



90% of cost in the cloud is for inference!



Design goal

- Utilize the expensive computing resources efficiently, so as to

- Minimize the total cost

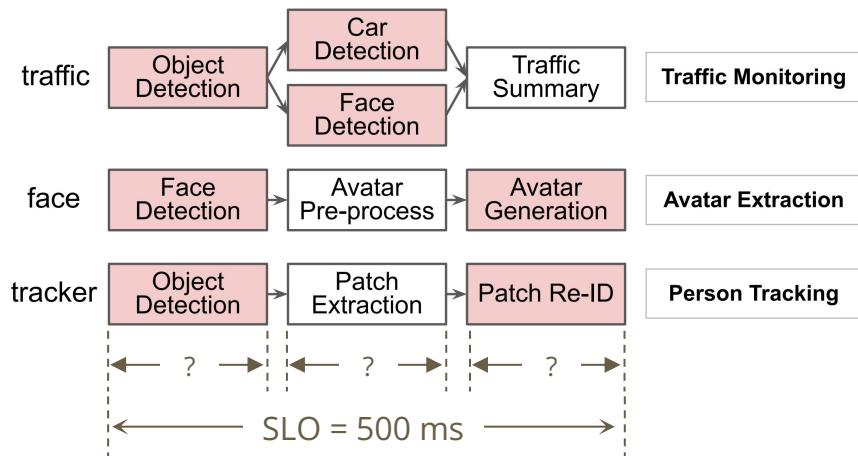


- Satisfy the latency objective



The general process of DNN serving

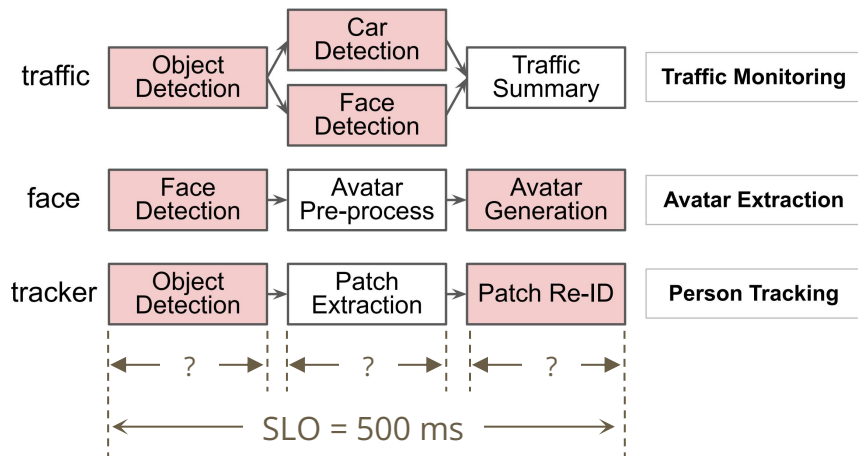
DNN pipeline: cascade of multiple models



Latency Splitting: split the end-to-end latency budget (SLO) into per module latency budget

The general process of DNN serving

DNN pipeline: cascade of multiple models



Latency Splitting: split the end-to-end latency budget (SLO) into per module latency budget

Cloud providers provide tens of VM types.

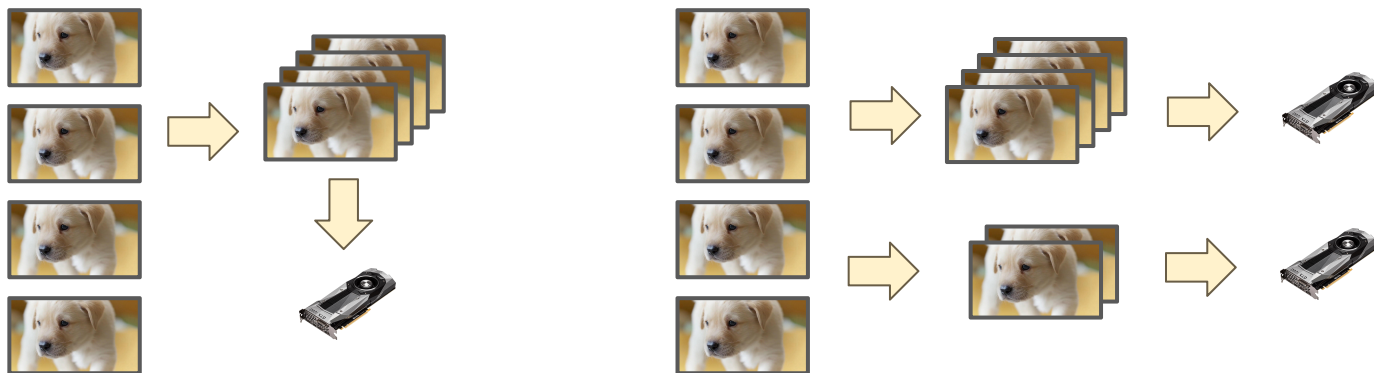
	P100	V100	Ratio
Pricing	2.07	3.06	1.5
SSD	14.7	16.3	1.1
YOLO	22.2	26.7	1.2

Throughput ratio varies across models

Module Scheduling: Select proper hardware and batching configs for each module

The general process of DNN serving

Batching: merge multiple requests into one batch to fully utilize the GPU.



Request Dispatch: Dispatch received requests to multiple GPU instances according to certain rules.

Design goal & Contributions

- Utilize the expensive computing resources efficiently, so as to

- Minimize the total cost



- Satisfy the latency objective



- Our contribution

- Request Dispatch: Lower Worst Case Latency

- Module Schedule: Higher module efficiency

- Latency Splitting: Higher latency-cost efficiency



minimize DNN
serving cost

How existing systems work?

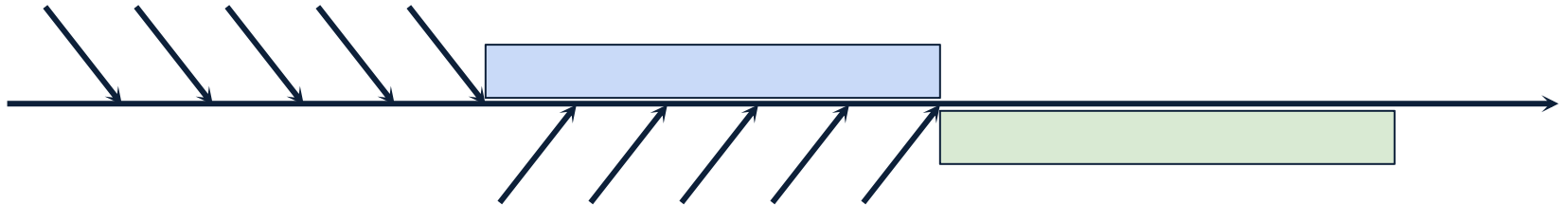
- Existing systems use a two-round heuristic to choose scheduling strategy.

Batch size	Duration	Throughput
5	0.10	50
20	0.25	80
100	1.00	100

How existing systems work?

- Existing systems use a two-round heuristic to choose scheduling strategy.

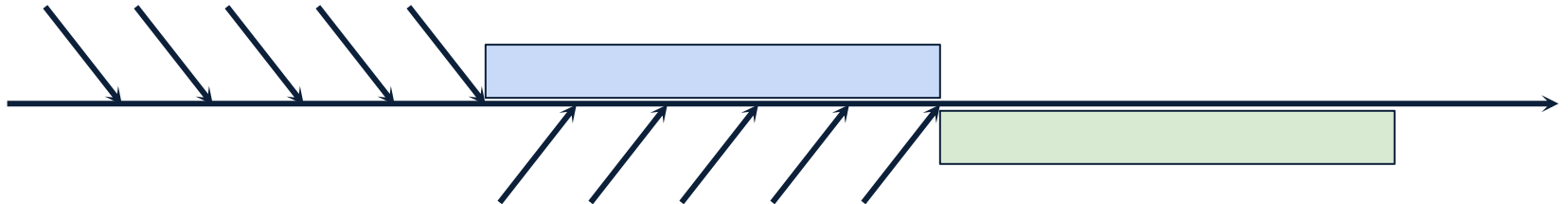
Batch size	Duration	Throughput
5	0.10	50
20	0.25	80
100	1.00	100



How existing systems work?

- Existing systems use a two-round heuristic to choose scheduling strategy.

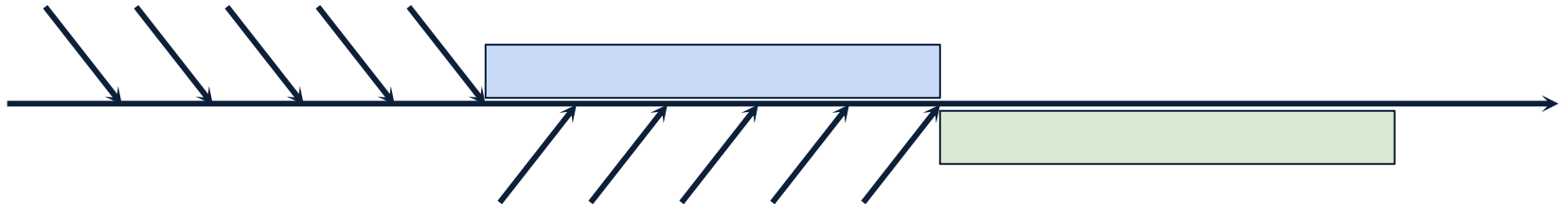
Batch size	Duration	Throughput	WCL = 2d
5	0.10	50	0.20
20	0.25	80	0.50
100	1.00	100	2.00



How existing systems work?

- Existing systems use a two-round heuristic to choose scheduling strategy.

Batch size	Duration	Throughput	WCL = 2d
5	0.10	50	0.20
20	0.25	80	0.50
100	1.00	100	2.00

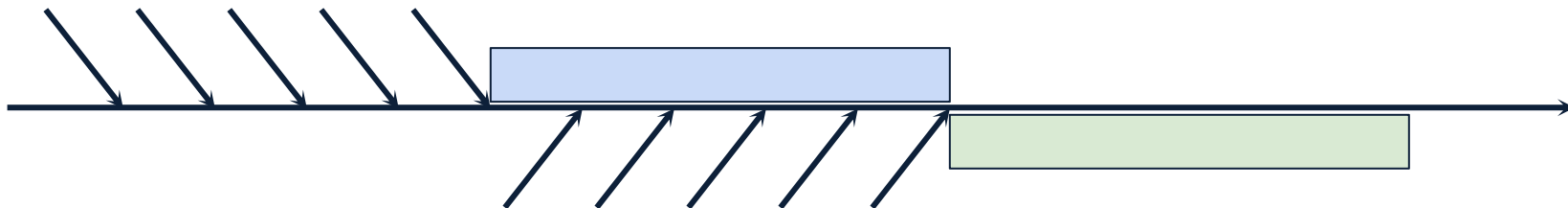


How existing systems work?

- Existing systems use a two-round heuristic to choose scheduling strategy.

Batch size	Duration	Throughput	WCL = 2d
5	0.10	50	0.20
20	0.25	80	0.50
100	1.00	100	2.00

When $L_{SLO} = 0.5$, batch size of 20 is optimal configuration.



How existing systems work?

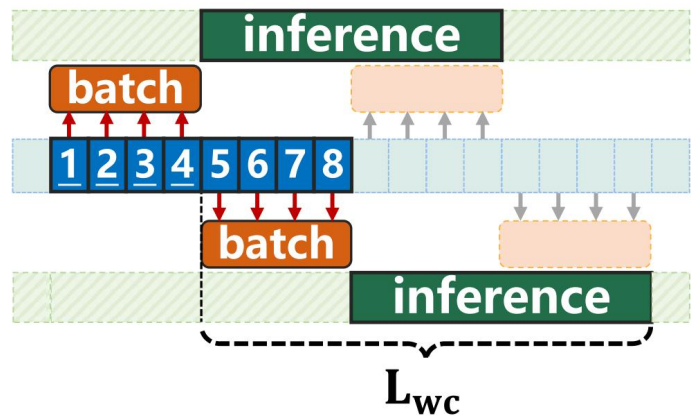
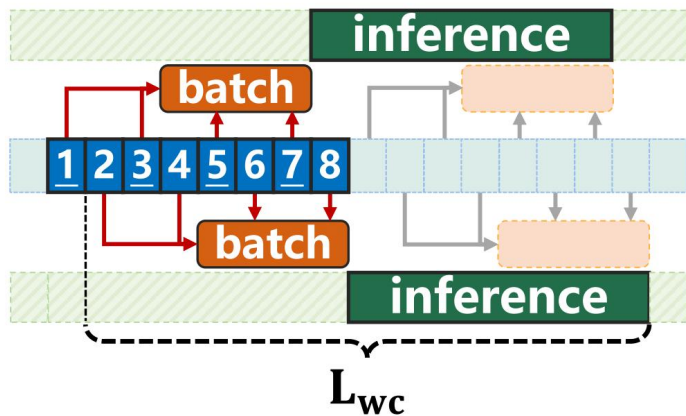
- Existing systems use a two-round heuristic to choose scheduling strategy.

Batch size	Duration	Throughput	WCL = 2d
5	0.10	50	0.20
20	0.25	80	0.50
100	1.00	100	2.00

- For $T = 285$, $L_{SLO} = 2.0$, existing systems will choose:
 - c_{opt} of batch size 100 for $T_{maj} = 200$
 - c_{res} of batch size 5 for $T_{res} = 85$

Q1: How to minimize worst case latency?

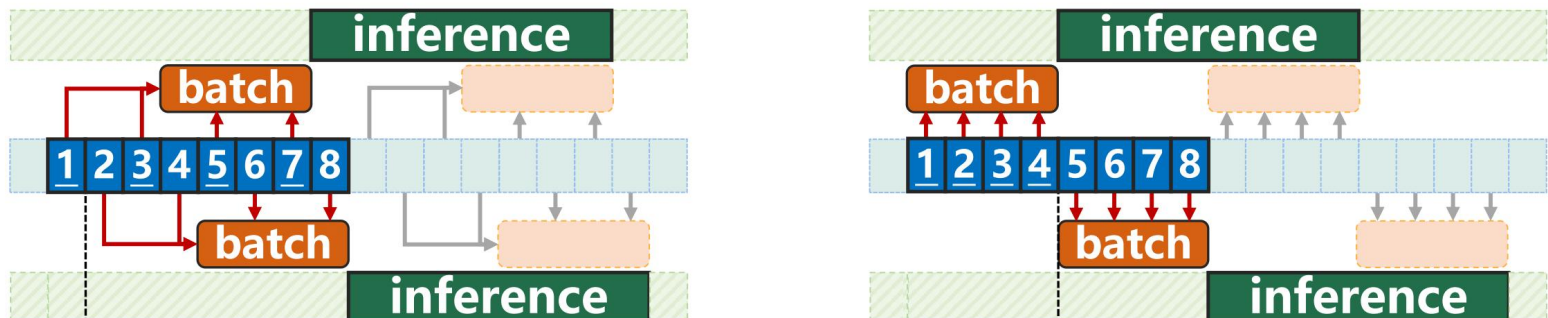
- Existing **round-robin dispatch** policy unnecessarily increases the WCL.



- WCL: $2d$ $\rightarrow$ $d+b/w=1.5d$
- The saved latency budget (ex. $0.5d$) can be used to reduce total cost.

Q1: How to minimize worst case latency?

- Existing **round-robin dispatch** policy unnecessarily increases the WCL.

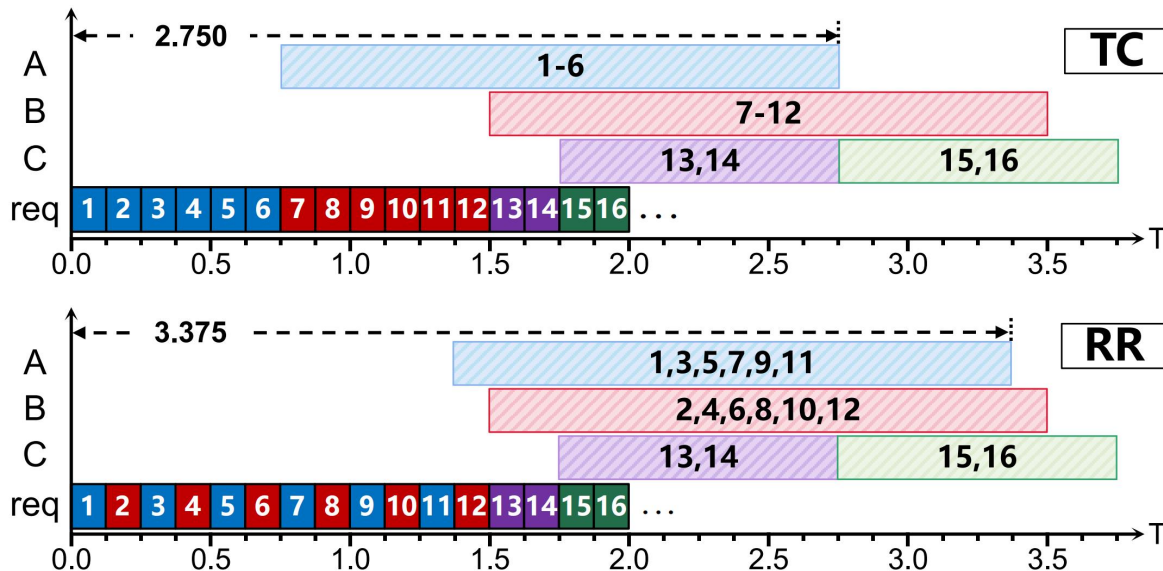


Insight 1:

Existing Round-Robin dispatch strategy introduces unnecessary latency to requests, leading to large L_{WC} .

Design: batch aware request dispatching

- Existing **round-robin dispatch** policy unnecessarily increases the WCL.
- Design: Batch aware request dispatching using **throughput-cost ratio**.



$$L_{wc} = \max\{d + b/w\}$$

$$L_{wc} = 2d$$

Q2: How to maximize module efficiency?

- Existing systems' **fixed of two configuration** can't minimize the cost.

Batch size	Duration	Throughput
5	0.10	50
20	0.25	80
100	1.00	100

- For $T = 285$, $L_{SLO} = 2.0$
 - Existing system: two machines w/ $b = 100$ and two machines w/ $b = 5$ $\implies$ cost = **3.7**
 - For residual, use one machine w/ $b = 20$ and one machine w/ $b = 5$ $\implies$ cost = **3.1**

Q2: How to maximize module efficiency?

- Existing systems' **fixed of two configuration** can't minimize the cost.

Insight 2:

Existing systems' fixed of two config scheduling policy cannot maximize module efficiency.

100	1.00	100
-----	------	-----

- For $T = 285$, $L_{SLO} = 2.0$
 - Existing system: two machines w/ $b = 100$ and two machines w/ $b = 5$ ==> cost = **3.7**
 - For residual, use one machine w/ $b = 20$ and one machine w/ $b = 5$ ==> cost = **3.1**

Design: two round configuration search

- Existing systems' **fixed of two configuration** can't minimize the cost.
- Design: Two round module configuration search w/ dummy request.

Method	S_1	S_2	S_3	S_4
Dispatch	round-robin	batch-aware	batch-aware	batch-aware
#Config	2	2	any	any
Dummy	X	X	X	✓
Config	192 ($6.0 \otimes 8$) 6 ($0.3 \otimes 2$)	160 ($4.0 \otimes 32$) 38 ($1.9 \otimes 2$)	160 ($4.0 \otimes 32$) 32 ($1.0 \otimes 8$) 6 ($0.3 \otimes 2$)	200 ($5.0 \otimes 32$)
Cost	6.3	5.9	5.3	5.0

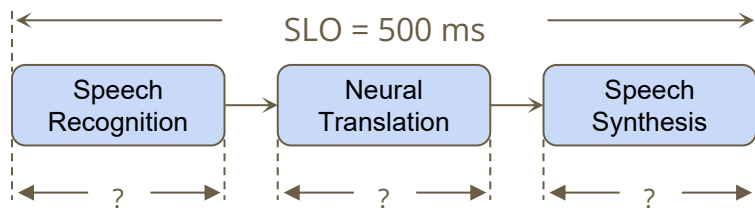
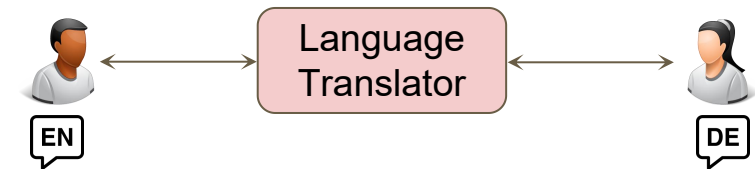
round 2 ($S_3 \rightarrow S_4$):
optimize the residual
workload with dummy
request

b	d	t
2	0.100	20
8	0.250	32
32	0.800	40

round 1 ($S_2 \rightarrow S_3$): throughput-cost
based configuration search

Q3: How to split request latency budget?

Applications with more than one DL models should be managed carefully.



NP-hard!

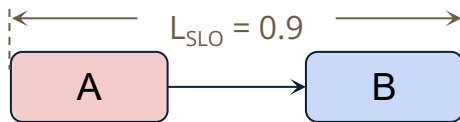
How to split e2e latency budget in reasonable time?



Q3: How to split request latency budget?

- Given an application (A $\rightarrow$ B), where $T_A = 50$, $T_B = 40$, $L_{SLO} = 0.9$

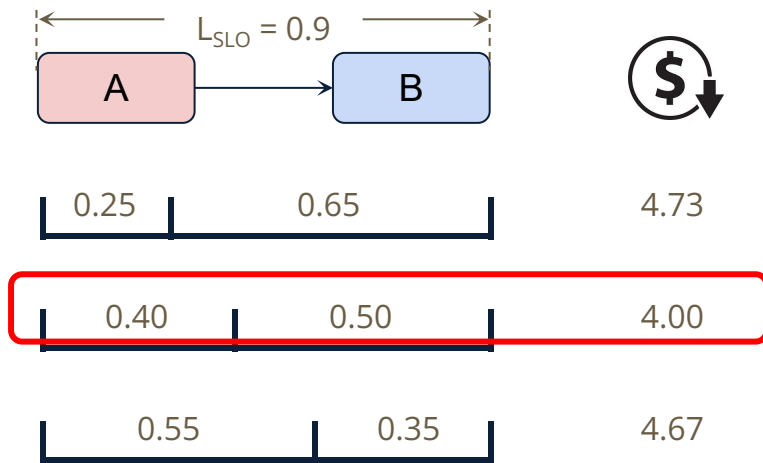
A			B		
b	d	t	b	d	t
2	0.125	16	2	0.167	12
4	0.160	25	4	0.200	20
8	0.267	30	8	0.320	25



Q3: How to split request latency budget?

- Given an application (A $\rightarrow$ B), where $T_A = 50$, $T_B = 40$, $L_{SLO} = 0.9$

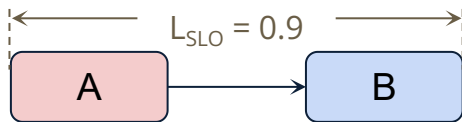
A			B		
b	d	t	b	d	t
2	0.125	16	2	0.167	12
4	0.160	25	4	0.200	20
8	0.267	30	8	0.320	25



Q3: How to split request latency budget?

- Given an application (A -> B), where $T_A = 50$, $T_B = 40$, $L_{SLO} = 0.9$

A			B		
b	d	t	b	d	t
2	0.125	16	2	0.167	12
4	0.160	25	4	0.200	20
8	0.267	30	8	0.320	25



4.73



4.00



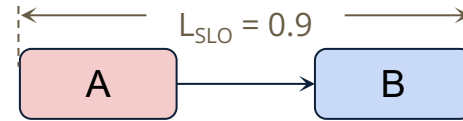
4.67



Q3: How to split request latency budget?

- Given an application (A $\rightarrow$ B), where $T_A = 50, T_B = 40$ $L_{SLO} = 0.9$

A			B		
b	d	t	b	d	t



Insight 3:

Existing systems' discretized split solution is too complex, and can still lead to suboptimal solution.



Design: iterative heuristic latency splitting

● Design: iterative heuristic latency splitting using latency-cost efficiency.

Algorithm 2 Deriving per-module latency budget

```
1: function SplitLatency( $T, L, P$ )
2:   DAG  $\leftarrow$  GetDefaultDAG()
3:   flag  $\leftarrow$  True
4:   while flag do
5:     flag  $\leftarrow$  False,  $LC_{max} \leftarrow -\infty$ 
6:      $M_{max} \leftarrow$  NULL,  $c_{max} \leftarrow$  NULL
7:     for  $M$  in DAG do
8:        $T_M \leftarrow T[M], P_M \leftarrow P[M]$ 
9:        $c_{prev} \leftarrow$  DAG[ $M$ ]
10:      for  $c_{new}$  in  $P_M$  do
11:         $LC \leftarrow \frac{p_{prev} \times T_M / t_{prev} - p_{new} \times T_M / t_{new}}{\text{GetWCL}(c_{new}) - \text{GetWCL}(c_{prev})}$ 
12:        if  $LC > LC_{max}$  then
13:          if  $\text{GetLat}(\text{DAG}, M, c_{new}) \leq L$  then
14:            flag  $\leftarrow$  True,  $LC_{max} \leftarrow LC$ 
15:             $M_{max} \leftarrow M, c_{max} \leftarrow c_{new}$ 
16:      if flag then
17:        updateDAG(DAG,  $M_{max}, c_{max}$ )
18:   return [ $L_M$  for  $M$  in DAG]
```

$$LC = \frac{C_M(\text{prev}) - C_M(\text{new})}{L_{WC}(\text{new}) - L_{WC}(\text{prev})}$$

Latency-cost efficiency (LC): amount of cost that a new config can reduce per unit of latency budget

Our solution

Insight 1: Existing Round-Robin dispatch strategy introduces unnecessary latency to requests, leading to large L_{wc} .



- Batch-aware dispatching
- Reducing L_{wc} from $2d$ to $\max\{d+b/w\}$

Insight 2: Existing systems' fixed of two config scheduling policy cannot maximize module efficiency.



- Heterogeneous multiple config
- Dummy generator for residual workload

Insight 3: Existing discretized latency budget splitting is too complex, and can still lead to suboptimal solution.

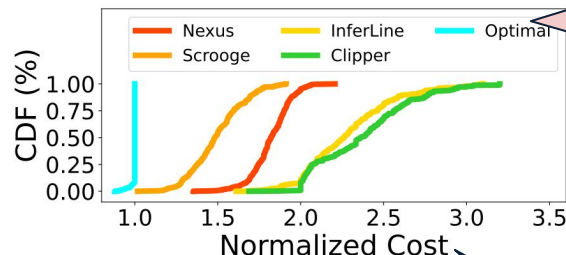
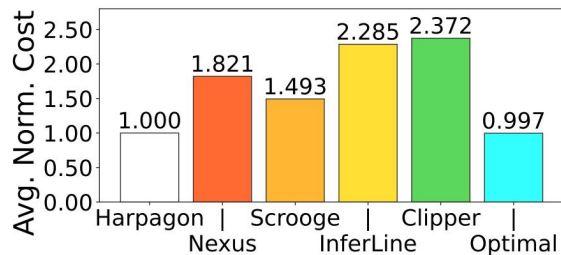


- Splitting w/ latency-cost efficiency
- Latency budget reassignment

Evaluation

COMPARISON OF SERVING SYSTEMS

System	Worst Case Latency	Num of Configurations	Batch	Hetero	Residual Optimization	Latency Split	Split Optimization
Harpagon	$d + b/w$	any	✓	✓	dummy + reassign	latency-cost efficiency	merge + direct
Nexus [2]	$2d$	2	✓			quantized interval	
Scrooge [3]	$d + b/t$	2	✓	✓		throughput-based	
InferLine [4]	$2d$	1	✓	✓		throughput-based	
Clipper [5]	$2d$	1	✓			evenly splitting	



Harpagon can save up to 2X cost.

Fig. 5. (a) The average normalized cost for Harpagon, four baseline systems, and the optimal solution under 1131 workloads; (b) CDF of normalized

Harpagon achieves optimal for 90+% workload.

Evaluation

COMPARISON OF SERVING SYSTEMS

System	Worst Case Latency	Num of Configurations	Batch	Hetero	Residual Optimization	Latency Split	Split Optimization
Harpagon	$d + b/w$	any	✓	✓	dummy + reassign	latency-cost efficiency	merge + direct
Nexus [2]	$2d$	2	✓			quantized interval	
Scrooge [3]	$d + b/t$	2	✓	✓		throughput-based	
InferLine [4]	$2d$	1	✓	✓		throughput-based	
Clipper [5]	$2d$	1	✓			evenly splitting	

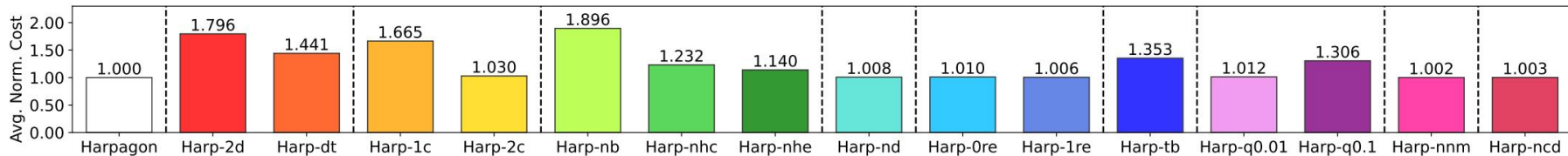


Fig. 6. The average normalized cost for Harpagon and baseline Harpagon without corresponding functionalities.

Takeaway

- Request dispatch policy affects the worst case latency.
- Multi-config and dummy requests can improve the cost efficiency.
- The optimal latency splitting strategy is decided by request rate.
- Harpagon can save up to 2 times the total cost.
- Harpagon achieves the optimal total cost for over 90% workload.



天津大学
Tianjin University



Harpagon: Minimizing DNN Serving Cost via Efficient Dispatching, Scheduling and Splitting

Zhixin Zhao¹, Yitao Hu¹, Ziqi Gong¹, Guotao Yang¹,
Wenxin Li¹, Xiulong Liu¹, Keqiu Li¹, Hao Wang²

zhixin612cs@gmail.com

¹ Tianjin Key Laboratory of Advanced Networking, Tianjin University

² Stevens Institute of Technology